

Cookies verstehen und anwenden

10 kleine Aufgaben · Startcode mit TODOs · Lösungen getrennt

Das Ziel

Den bekannten Formularablauf um einen kleinen Speicher im Browser erweitern: Daten prüfen, Cookie setzen, zurückleiten und beim nächsten Request geprüft auslesen. Keine Sessions, keine Datenbank, kein Login.

Start in XAMPP

Lege das gesamte Repository PHP_AI_DEMO unter C:\xampp\htdocs ab und starte Apache. Öffne die Kursübersicht unter http://localhost/PHP_AI_DEMO/ und wähle den passenden Arbeitsbereich.

Einstieg im Repository: [PHP_Arbeitsbereich/cookies/index.html](#)

Drei bekannte Dateien

index.php zeigt das Formular und liest Cookies. **auswertung.php** prüft POST-Daten, setzt oder löscht Cookies und leitet weiter. **function.php** enthält den vorgegebenen HTML-Helper und den URL-Ordnerpfad. Die Aufgaben 7 und 8 ergänzen eine zweite Anzeigeseite.

Für jede Aufgabe verbindlich

1. Prüfe jeden POST- und Cookie-Eintrag vor dem Auslesen mit `isset()` und `is_string()`. Prüfe danach, ob der Inhalt erlaubt ist. [2, 7]
2. Vor `setcookie()` und `header()` keine Ausgabe. Nach der Rückleitung folgt `exit`. Verwende für die Rückleitung HTTP 303. [1, 3, 6]
3. Bei einem Formfehler kein Cookie verändern. Gültige neue Formwerte erhalten, ungültige Felder leeren. Ältere gespeicherte Werte sind ein anderer Zustand.
4. Nur harmlose Einstellungen speichern. Alle Beispiele laufen lokal mit HTTP; `secure` ist deshalb `false`. Bei HTTPS `secure` aktivieren. [5]
5. Jeder Aufgabenordner hat eigene Cookie-Namen und einen eigenen Pfad. Name und Pfad beim Setzen und Löschen beibehalten. [4]

Arbeitsweise

Zuerst Webeinführung und Demo ansehen. Danach die TODO-Stellen in `aufgaben/` bearbeiten. Die HTML-Formulare sind fertig vorbereitet; neu zu schreiben sind vor allem Prüfung, Cookie-Aktion und Cookie-Empfang. Die Musterlösungen erst zum Abgleich öffnen.

01 Eine Ansicht merken

POST · Ansicht: ansicht

Cookie setzen und nach der Rückleitung auslesen

Cookie: PHP_c01_anzicht

1. Sende ansicht mit POST. Prüfe mit `isset()`, ob der Eintrag existiert, und mit `is_string()`, ob ein Textwert vorliegt. Erlaubt sind nur liste und kacheln.
2. Bei einem Fehler: zurück zum Formular, Fehlermeldung zeigen und kein Cookie setzen.
3. Speichere die gültige Ansicht für eine Stunde im vorgegebenen Cookie. Leite danach mit HTTP 303 zu `index.php` zurück und beende das Skript.
4. Lies das Cookie in `index.php` erst nach Prüfung aus. Zeige die Ansicht an und wähle sie im Formular wieder aus. Ohne gültiges Cookie gilt liste.

Teste selbst

Kacheln wählen → nach der Rückleitung Kacheln; F5 → weiter Kacheln.

Keine Auswahl oder ansicht als Array senden → Fehler; kein neues Cookie.

Cookie im Browser löschen → Standard Liste; keine PHP-Warnung.

02 Das Farbschema ändern

POST · Farbschema: schema

Ein vorhandenes Cookie gezielt überschreiben

Cookie: PHP_c02_schema

1. Empfange schema mit POST. Prüfe Vorhandensein, Texttyp und die erlaubten Werte hell oder dunkel.
2. Speichere eine gültige Auswahl für einen Tag. Verwende bei jeder Änderung denselben Cookie-Namen und denselben Pfad.
3. Leite nach dem Setzen zurück. Lies das Cookie geprüft ein; ohne gültigen Wert gilt hell.
4. Verwende die geprüfte Auswahl für das Formular und die vorbereitete Darstellung. Ungültige Eingaben dürfen ein vorhandenes Cookie nicht überschreiben.

Teste selbst

Hell speichern, danach Dunkel → genau ein Cookie mit dem neuen Wert.

F5 → weiterhin Dunkel.

schema=bunt senden → Fehler; gespeichertes Farbschema bleibt unverändert.

03 Eine Einstellung löschen

POST · Schriftgröße: schrift

Cookie im Browser entfernen, nicht nur in PHP

Cookie: PHP_c03_schrift

1. Verarbeite die POST-Aktion speichern oder loeschen. Beim Speichern ist schrift Pflicht; erlaubt sind normal und gross.
2. Speichere eine gültige Schriftgröße für eine Stunde. Nach der Rückleitung wird sie angezeigt und ausgewählt.
3. Ergänze den Knopf „Cookie löschen“. Sende dafür das Cookie mit einem Ablaufzeitpunkt in der Vergangenheit. Name und Pfad bleiben identisch.
4. Leite auch nach dem Löschen zurück. Ohne Cookie erscheint Normal. Die Löschaktion darf keine neue Schrift-Auswahl verlangen.

Teste selbst

Groß speichern → Groß wird angezeigt.

Ohne Auswahl „Cookie löschen“ → Cookie weg; Standard Normal.

Erneut löschen → weiterhin ohne Cookie; keine PHP-Warnung.

04 Nur kurz merken

POST · Motiv: motiv

Ablaufzeitpunkt und erneute Anfrage unterscheiden

Cookie: PHP_c04_motiv

1. Prüfe die POST-Eingabe motiv. Erlaubt sind stern und kreis; fehlende, leere oder andere Werte erzeugen einen Fehler.
2. Merke die gültige Auswahl für 30 Sekunden. Setze das Cookie ausschließlich beim erfolgreichen Speichern, nicht bei jedem GET-Aufruf.
3. Leite zurück. Zeige neben der Auswahl an, ob im aktuellen Request ein gültiges Cookie angekommen ist.
4. Warte länger als 30 Sekunden und lade die Seite neu. Probiere danach expires = 0 aus und notiere den Unterschied. Das ist eine Browser-Sitzung, noch keine PHP-Session.

Teste selbst

Kreis speichern → Cookie angekommen.

Nach etwa 10 Sekunden F5 → Ablaufzeit wird nicht verlängert.

Nach mehr als 30 Sekunden F5 → kein Cookie, Standard Stern; die alte HTML-Seite ändert sich nicht von allein.

05 Cookie-Werten nicht blind vertrauen

POST · Einträge pro Seite: anzahl

Auch gespeicherte Werte auf dem Server validieren

Cookie: PHP_c05_anzahl

1. Prüfe anzahl beim POST-Empfang. Erlaubt sind ausschließlich die Textwerte 5, 10 und 20.
2. Speichere eine gültige Auswahl für eine Stunde und leite zurück.
3. Prüfe in index.php zusätzlich das empfangene Cookie: vorhanden, Texttyp und erlaubter Wert. Bei einem fremden Wert gilt 10; zeige einen Hinweis.
4. Ändere den Cookie-Wert über die Browser-Entwicklerwerkzeuge auf 999 und lade neu. Verwende einen ungeprüften Cookie-Wert weder für die Ausgabe noch für eine Berechnung.

Teste selbst

20 speichern → 20 wird verwendet.

Cookie auf 999 ändern und F5 → Hinweis und Standard 10.

Cookie löschen → Standard 10 ohne Manipulationswarnung; POST-Wert 7 → Eingabefehler.

06 Lautstärke null ist gültig

POST · Lautstärke (0 bis 10): lautstaerke

Zahlen prüfen, ohne den Wert 0 zu verlieren

Cookie: PHP_c06_lautstaerke

1. Empfange lautstaerke mit POST. Prüfe isset() und is_string(); entferne äußere Leerzeichen.
2. Erlaubt ist eine ganze Zahl von 0 bis 10. Eine leere Eingabe ist ungültig, die Zahl 0 ist ausdrücklich gültig.
3. Speichere nur eine gültige Zahl für eine Stunde und leite zurück. Auch die empfangene Cookie-Zahl wird erneut geprüft.
4. Ohne gültiges Cookie gilt 5. Bei einem Formfehler bleibt das Eingabefeld leer; ein vorher gültiges Cookie wird nicht verändert.

Teste selbst

0 speichern und F5 → weiterhin 0.

Leer, -1, 11 oder 2.5 → Fehler und kein neues Cookie.

Cookie auf abc ändern → Hinweis und Standard 5.

07 Eine zweite Seite liest mit

POST · Sprache: sprache

Ein Cookie bei mehreren Seitenaufrufen verwenden

Cookie: PHP_c07_sprache

1. Prüfe sprache im POST-Formular. Erlaubt sind de und en. Speichere einen gültigen Wert für eine Stunde im vorgegebenen Ordnerpfad.
2. Leite nach dem Speichern zu index.php zurück. Zeige dort die geprüfte Auswahl.
3. Ergänze anzeige.php im selben Ordner. Diese Seite erhält keine Sprache über GET oder POST, sondern liest ausschließlich das geprüfte Cookie.
4. Zeige bei de „Hallo“ und bei en „Hello“. Ohne gültiges Cookie gilt de. Ergänze einen Link zurück zum Formular.

Teste selbst

Englisch speichern → anzeige.php zeigt Hello, obwohl die URL keine Sprache enthält.

anzeige.php direkt öffnen → Cookie wird trotzdem verwendet.

Cookie löschen, anzeige.php neu laden → Hallo.

08 Den Cookie-Pfad eingrenzen

POST · Sortierung: sortierung

Ein URL-Pfad ist kein Dateisystempfad

Cookie: PHP_c08_sortierung

1. Prüfe sortierung per POST. Erlaubt sind titel und datum. Ergänze den vorbereiteten Cookie-Pfad so, dass das Cookie nur für bereich/ und dessen Unterpfade gilt.
2. Speichere für eine Stunde und leite zu bereich/anzeige.php. Dort wird das Cookie geprüft und die Sortierung angezeigt.
3. Öffne anschließend index.php im übergeordneten Aufgabenordner. Dort soll dieses Cookie nicht im Request ankommen; das ist kein Speicherfehler.
4. Ergänze die Löschaktion in auswertung.php. Lösche mit genau dem eingeschränkten Pfad und leite zurück zu bereich/anzeige.php.

Teste selbst

Datum speichern → in bereich/anzeige.php kommt Datum an.

Link zum übergeordneten Formular → dort kein passendes Cookie im Request.

Cookie löschen → auch im Bereich kein Cookie mehr; Standard Titel.

09 Nur auf Wunsch merken

POST · Startbereich: startbereich · Checkbox: merken (ja)

Checkbox, Vorschau und dauerhafte Auswahl trennen

Cookie: PHP_c09_startbereich

1. Empfange startbereich und die freiwillige Checkbox merken. Startbereich muss kurse oder termine sein. Die Checkbox darf fehlen; vorhanden ist nur der Textwert ja gültig.
2. Prüfe beide Angaben vor jeder Cookie-Änderung. Bei einem Fehler: alle Meldungen sammeln, gültige Formwerte erhalten und kein Cookie ändern.
3. Mit Haken: Startbereich für einen Tag merken. Ohne Haken: ein eventuell früher gesetztes Cookie mit demselben Namen und Pfad löschen.
4. Leite zurück. Ohne Haken darf die gültige Auswahl für die aktuelle Vorschau über GET zurückkommen. Ein frischer Aufruf ohne URL-Parameter verwendet aber nur das Cookie oder den Standard kurse.

Teste selbst

Termine + Haken → frischer Aufruf merkt Termine.

Termine ohne Haken → aktuelle Vorschau Termine, frischer Aufruf Kurse; Cookie weg.

Leerer Startbereich + Haken → Fehler, Haken bleibt; fremder Checkbox-Wert → Fehler.

10 Zwei Einstellungen gemeinsam speichern

POST · Sprache: sprache · Darstellung: darstellung

Alles prüfen, dann Cookies setzen

Cookie: PHP_c10_sprache

1. Empfange sprache und darstellung mit POST. Prüfe beide Einträge mit isset() und is_string(). Erlaubt sind de/en sowie kompakt/ausfuehrlich.
2. Prüfe beide Felder vollständig. Bei Fehlern: Meldungen sammeln, gültige Formwerte wieder einsetzen und ungültige leeren. Kein Cookie darf dabei verändert werden.
3. Erst wenn beide Werte gültig sind, setze zwei getrennte Cookies für einen Tag. Verwende für beide denselben vorgegebenen Pfad und leite zurück.
4. Lies beide Cookies unabhängig voneinander geprüft ein. Standards: de und kompakt. Unterscheide im Fehlerfall die aktuell eingegebenen Werte von älteren, weiterhin gespeicherten Einstellungen.

Teste selbst

Englisch / Ausführlich → zwei Cookies, beide Werte nach frischem Aufruf erhalten.

Englisch / leer → Englisch im Formular, Darstellung leer, Fehlermeldung; keine Cookie-Änderung.

Beide leer → zwei Meldungen; ein Cookie manipulieren → nur dessen Standard verwenden.

Merkhilfe

Bausteine und Denkregeln – keine Aufgabenlösungen.

Wer macht was?

PHP sendet Set-Cookie in der Antwort. Der Browser speichert, sofern erlaubt. Erst eine spätere passende Anfrage sendet Cookie zurück; PHP stellt die Werte in `$_COOKIE` bereit. [1, 2, 4]

Die vier Zustände auseinanderhalten

Nicht vorhanden ist nicht dasselbe wie leer, gültig oder ungültig. Erst `isset()`, dann Texttyp und Inhalt prüfen. Bei fehlenden oder unzulässigen Cookie-Werten einen Standard verwenden. [2, 7]

Zeit richtig lesen

`time() + 3600`: eine Stunde ab jetzt. `time() + 30`: 30 Sekunden. `time() - 3600`: abgelaufen.
`expires = 0`: Browser-Sitzung, nicht sofort löschen. Eine Sitzungswiederherstellung kann solche Cookies erhalten. [1, 4]

Pfad und Löschen

path ist ein URL-Bereich, z. B. `/PHP _Cookies/demo/`. Kein Windows-Ordnerpfad. Zum Löschen Name, Pfad und eine eventuell gesetzte Domain beibehalten. `unset()` allein löscht kein Browser-Cookie. [2, 4, 5]

Nicht verwechseln

Ein Session-Cookie ist noch keine PHP-Session. `HttpOnly` verhindert JavaScript-Zugriff, aber keine Änderungen durch den Nutzer. Cookie-Werte bleiben Client-Eingaben. Die Schutzattribute ersetzen keine fachliche Validierung. [4, 5]

So testest du

F12 → Anwendung / Application → Cookies. Werte und Pfad ansehen. Für Request und Response im Netzwerk das Protokoll beibehalten. Set-Cookie und Location in der 303-Antwort; Cookie im Folge-Request prüfen. [5, 6, 9]

Technische Referenzen

[1] PHP: `setcookie()`

[2] PHP: `$_COOKIE`

[3] PHP: `header()`

[4] MDN: HTTP-Cookies

[5] MDN: Set-Cookie-Header

[6] MDN: HTTP 303 See Other

[7] PHP: `isset()`

[8] PHP: `htmlspecialchars()`

[9] Chrome DevTools: Cookies untersuchen

Neu erstellte Übungen. Technische Grundlage: PHP-Handbuch und Browserdokumentation; vollständige Links stehen auch in der Webvariante.